π-FORMER: SUCCINCT PROOFS OF TRANSFORMER INFERENCE VIA STRUCTURED LOOKUP ATTENTION

Hideaki Takahashi
Department of Computer Science
Columbia University
ht2673@columbia.edu

ABSTRACT

We present π-Former, a succinct SNARK system for verifiable transformer inference, in which the model architecture and the proof system are co-designed so that every operation in a forward pass maps to a low-degree sumcheck claim or a structured table lookup. The architecture replaces the three classical SNARK bottlenecks of a transformer block: softmax becomes *Learnable Lookup Attention* (LLA), a linear attention whose feature map is parametrised as a sum of small Lasso-compatible sub-tables; layer normalisation is replaced by a *constraint-fused* circuit that proves the normalisation identity at one random point with chunked range proofs and contains no field divisions; and dense projections are restricted to ternary weights $\{-1, 0, +1\}$ scaled by a single learnable α, so that every contraction is a sum of additions and subtractions with no field multiplications.

The proof system is built around *cross-block batching*: rather than producing one sub-proof per protocol per block, the prover runs six *model-level* batched sumchecks (QKV projection, output projection, attention-output, attention-context, FFN-Y, FFN-M) that fold the per-block claims via Fiat–Shamir powers of an independent challenge. All Lasso instances across all blocks fold into one global Lasso proof; all LayerNorm range witnesses share one multiplicity commitment; residual connections are checked by homomorphic addition of Hyrax commitments (no proof at all); and all final Hyrax openings flow through deferred batch accumulators that finalise into one MSM pair per group. We instantiate the system over BN254 with the Hyrax polynomial commitment scheme, giving a fully transparent setup.

We implement π-Former in $\sim$14 k lines of Rust with a PyTorch training pipeline whose integer forward pass matches the Rust circuit's arithmetic exactly. On a single x86_64 CPU core, end-to-end proving for a 4-layer model with d=128, d_{ff}=512, T=32 ($\sim$924 k parameters) takes 4.2 s; the verifier's measured end-to-end compute is 280 ms and the proof is 5.5 MB. We give formal cross-block batching soundness theorems and bound the end-to-end soundness error by $((m_{\mathrm{sc}} n_{\max} \delta_{\max} + m_{\mathrm{batch}}(L-1) + m_{\mathrm{open}})/|\mathbb{F}|) + \varepsilon_{\mathrm{DL}}$.

1 Introduction

Large language models are increasingly deployed in settings such as legal review, medical decision support, and financial auditing, where the user needs cryptographic assurance that a claimed output was produced by a specific committed model on a specific input. Succinct non-interactive arguments provide the right primitive: the model owner produces a compact proof, and the verifier checks it without executing the model or reading committed proprietary weights. Zero-knowledge masking is orthogonal to this integrity goal and is not enabled in the current prototype [8, 13, 1].

The bottleneck is concrete cost. Transformers are built from exactly the operations that finite-field arithmetic circuits find expensive: softmax requires exponentials and divisions, GeLU is transcendental, layer normalisation contains a square root and a division by a running variance, and a single dense linear projection costs one field multiplication per weight entry. State-of-the-art softmax-aware systems [13, 10] introduce careful approximations and large lookup tables to handle these operations; the resulting prover times for GPT-scale models are still measured in minutes per token.

Our approach. π-Former starts from a different premise: *co-design* the model and the proof system so that every operation in inference maps to a low-degree sumcheck claim or a structured Lasso lookup. The three modelling ingredients are:

1. *Learnable Lookup Attention (LLA)*, a linear attention [6, 17] whose feature map ϕ is a learnable sum of c sub-tables of size $2^{b/c}$, exactly matching Lasso's structured-table condition;
2. *Ternary projections* that absorb all magnitude into one per-layer scalar α and reduce a contraction to additions and subtractions [9];
3. a *constraint-fused LayerNorm* circuit that contains no divisions or square roots and is verified at a single random index per layer.

Together they reduce the entire transformer block to operations a SNARK can prove with sub-linear verifier work.

Cross-block batching. The proof system itself is engineered around a single observation: *the same protocol is run once per block, with independent inputs but identical structure.* π-Former exploits this by running six *model-level* batched sumchecks (one per protocol type), folding all L per-block claims via a Fiat–Shamir random linear combination in an independent challenge η; the inner sumcheck transcript is shared across all blocks. The same idea applies to Lasso (one global multi-instance proof), to range checks (one shared multiplicity commitment), to residuals (homomorphic addition of Hyrax commitments; no proof at all), and to the final stage of opening (10 cross-cutting deferred Hyrax accumulators, each finalising in one MSM pair). None of these is a soundness-affecting trick; each is a standard random-linear-combination argument whose error is dominated by the inner sumcheck error.

Threat model and scope. We target the standard *non-interactive argument of knowledge* setting in the random-oracle model. The verifier is given a verifying key vk binding the model weights and lookup tables, a public input $\mathbf{x}_{\text{in}}$, and a claimed output $\mathbf{y}$. π-Former's prover is computationally bounded; its goal is to prove $\mathcal{M}_\theta(\mathbf{x}_{\text{in}}) = \mathbf{y}$. Soundness is computational under the discrete-log assumption on BN254 G1 (the underlying assumption for the Hyrax polynomial commitment scheme [16]). The current implementation is *not* zero-knowledge: weight commitments are public VK entries and intermediate sumcheck evaluations are revealed. We therefore use "verifiable inference" to refer to integrity, not witness privacy, and describe the standard masking extension in §8.

Contributions.

1. We design a transformer block (§3) in which every operation, attention, normalisation, projection, has a sub-linear verifier circuit, by combining LLA, ternary projections, and constraint-fused LayerNorm.
2. We give a complete end-to-end SNARK protocol (§4) that batches all L per-block sub-proofs into six model-level cross-block sumchecks plus one global Lasso, one global range batch, and ten deferred Hyrax accumulators (Algorithms 1–7).
3. We prove cross-block batching soundness (§5, Theorem 5.3) and bound the end-to-end soundness error (Theorem 5.5).
4. We implement π-Former in $\sim$14 k lines of Rust with a matched PyTorch training pipeline (§6) and evaluate it on three model sizes; for the 4-layer, $\sim$924 k-parameter model we report 4.2 s prove and 280 ms verify with a 5.5 MB proof (§7).

2 Background

2.1 Notation

We work over the BN254 scalar field $\mathbb{F} = \mathbb{F}_r$ with $r \approx 2^{254}$, with the Hyrax PCS instantiated on the BN254 G1 group $\mathbb{G}$ of the same prime order. We write L for the number of transformer blocks, T for the sequence length, d for the embedding dimension, d_h for the per-head dimension (we focus on single-head; $d_h = d$), and d_{ff} for the FFN width. A multilinear polynomial $\widetilde{f}$ on n variables is identified with its 2^n evaluations on $\{0,1\}^n$; its evaluation at $r \in \mathbb{F}^n$ is denoted $\widetilde{f}(r)$. The *equality polynomial* $\widetilde{\mathrm{eq}}(r,x) = \prod_{i=1}^{n}(r_i x_i + (1-r_i)(1-x_i))$ satisfies $\sum_x \widetilde{\mathrm{eq}}(r,x)\widetilde{f}(x) = \widetilde{f}(r)$. We use Com for Hyrax commit, Open for an opening proof, and FS for a Fiat–Shamir transcript operation. Capital bold $\mathbf{X}, \mathbf{W}, \ldots$ denote matrices represented as multilinear polynomials over $\mathbb{F}$ after fixed-point quantisation (§2.6).

2.2 The transformer block

A standard pre-norm transformer block [15] computes

$$\mathbf{X}_{\text{mid}} = \mathbf{X}_{\text{in}} + \text{Att}\big(\text{LN}(\mathbf{X}_{\text{in}})\big),$$
$$\mathbf{X}_{\text{out}} = \mathbf{X}_{\text{mid}} + \text{FFN}\big(\text{LN}(\mathbf{X}_{\text{mid}})\big),$$

where standard self-attention is $\text{Att}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^\top/\sqrt{d})\mathbf{V}$ and layer normalisation [3] is $\text{LN}(\mathbf{x}) = \boldsymbol{\gamma} \odot (\mathbf{x} - \mu)/\sqrt{\sigma^2 + \epsilon} + \boldsymbol{\beta}$. Each of softmax, GeLU and LayerNorm contains operations (exp, division, square root) without compact polynomial representations, so they dominate SNARK complexity.

2.3 The sumcheck protocol

For a multilinear polynomial $f : \mathbb{F}^n \to \mathbb{F}$ of small individual degree δ, the sumcheck protocol [11, 14] reduces a claim $H = \sum_{x \in \{0,1\}^n} f(x)$ to a single evaluation $f(r)$ at a random $r \in \mathbb{F}^n$ in n rounds; in each round the prover sends a univariate polynomial of degree $\leq \delta$, the verifier spot-checks consistency, and a Fiat–Shamir-derived random challenge r_i folds the polynomial. The protocol has soundness error $\leq n\delta/|\mathbb{F}|$ and a verifier cost of $O(n\delta)$ field operations. We use the degree-2 variant for products of two MLEs, the cubic variant inside LayerNorm, and a multi-batched variant $\sum_k \lambda_k \sum_x f_k(x) g_k(x)$ for cross-block batching.

2.4 Hyrax PCS

For a multilinear polynomial of $2^n = 2^{\nu+\sigma}$ evaluations arranged as a $2^\nu \times 2^\sigma$ matrix M, Hyrax commits to each row as a vector Pedersen commitment $C_i = \text{MSM}(\mathbf{g}, M[i])$ over public generators $\mathbf{g}$ derived as $g_j = \text{SHA3}(\texttt{"piformer-hyrax-gen"} \| j) \cdot G$, and opens at $r = (r_L \| r_R)$ by sending the row-collapsed vector $w'_j = \sum_i L_i(r_L) \cdot M[i][j]$. The verifier checks (i) a homomorphic commitment identity $\sum_i L_i(r_L) C_i = \text{MSM}(\mathbf{g}, w')$ and (ii) an inner-product identity $\langle R(r_R), w' \rangle = v$ [16]. Hyrax is transparent (no trusted setup), computationally binding under DL on $\mathbb{G}$, and has $O(\sqrt{N})$ commitments, opens in $O(\sqrt{N})$ G1 operations, and produces $O(\sqrt{N})$ proof bytes.

2.5 Lasso lookup arguments

Given a sub-table $T \in \mathbb{F}^{2^m}$ and queries $\{(\text{idx}_j, v_j)\}_{j=1}^N$ with $v_j = T[\text{chunk}(\text{idx}_j)]$, Lasso [12, 2] proves all queries are consistent with T via a single batched sumcheck for $\sum_x \widetilde{T}(x) \cdot L(x) = \sum_j \rho^j T[\text{ch}_j]$ where $L(x) = \sum_j \rho^j \widetilde{\text{eq}}(\text{bin}(\text{ch}_j), x)$ and ρ is Fiat–Shamir-derived. A composite table that decomposes *additively* into c sub-tables (the canonical Lasso "structured" condition) is committed as c small tables of size $2^{b/c}$ instead of one monolithic table of size 2^b.

2.6 Fixed-point encoding

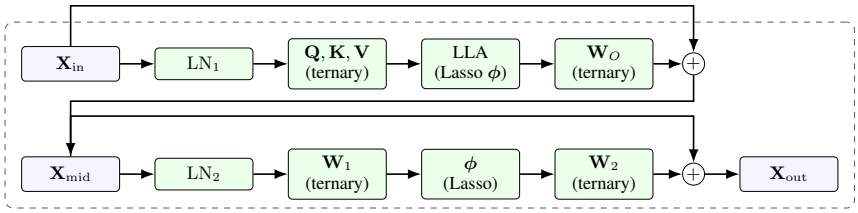
All real-valued tensors are quantised to integers and embedded in $\mathbb{F}$ by $x_{\mathbb{F}} = \lfloor x/s \rfloor \bmod r$, with the scale s chosen so no intermediate value overflows r (e.g., for b-bit activations and k-bit weights, $T \cdot d_h \cdot 2^b \cdot 2^k \ll r$). The PyTorch training pipeline applies the *same* integer arithmetic in its forward pass, so the exported witness exactly matches the field arithmetic in the Rust circuit, eliminating the train–prove gap.

3 The π-Former Architecture

A π-Former block (Figure 1) is a pre-norm linear-attention block. Let $\mathbf{X}_{\text{in}} \in \mathbb{F}^{T \times d}$ denote the input hidden state; the block computes

$$\mathbf{X}_{\text{n1}} = \text{LN}(\mathbf{X}_{\text{in}}; \boldsymbol{\gamma}_1, \boldsymbol{\beta}_1), \tag{1}$$
$$\mathbf{Q}, \mathbf{K}, \mathbf{V} = \mathbf{X}_{\text{n1}} \mathbf{W}_{Q,K,V} \ \text{(ternary, batched sumcheck)}, \tag{2}$$
$$\mathbf{O}_{\text{attn}} = \text{LLA}(\mathbf{Q}, \mathbf{K}, \mathbf{V})\, \mathbf{W}_O \ \text{(linear attention + output proj)}, \tag{3}$$
$$\mathbf{X}_{\text{mid}} = \mathbf{X}_{\text{in}} + \mathbf{O}_{\text{attn}} \ \text{(homomorphic residual)}, \tag{4}$$
$$\mathbf{X}_{\text{n2}} = \text{LN}(\mathbf{X}_{\text{mid}}; \boldsymbol{\gamma}_2, \boldsymbol{\beta}_2), \tag{5}$$
$$\mathbf{O}_{\text{ffn}} = \boldsymbol{\phi}(\mathbf{X}_{\text{n2}} \mathbf{W}_1)\, \mathbf{W}_2 \ \text{(structured-lookup FFN)}, \tag{6}$$
$$\mathbf{X}_{\text{out}} = \mathbf{X}_{\text{mid}} + \mathbf{O}_{\text{ffn}}. \tag{7}$$

After L such blocks the model applies one final LayerNorm and a ternary language-model head.



Cross-block batching: boxes of one type across all L blocks fold into one model-level proof: **ternary** → `qkv`/`oproj`/`ffn` sumchecks; **LLA inner** → attention out/context sumchecks; **Lasso** $\boldsymbol{\phi}$ → one global multi-proof; **LN range** → one shared multiplicity commitment; $+$ → homomorphic Hyrax addition.

Figure 1: A single π-Former block. Boxes are matrices; rounded green nodes are operations the SNARK proves. "Ternary" projections contain no field multiplications inside the contraction; "Lasso" operations are proved through structured table lookups; residual $+$'s are checked by homomorphic addition of Hyrax commitments and require no proof. The cross-block batching architecture (§4) collapses each protocol type across all L blocks into a single model-level proof.

3.1 Learnable Lookup Attention (LLA)

Standard softmax attention requires n^2 exponentials and n row-wise divisions. We replace the kernel with a positive-definite inner product $\kappa(\mathbf{q}, \mathbf{k}) = \langle \boldsymbol{\phi}(\mathbf{q}), \boldsymbol{\phi}(\mathbf{k}) \rangle$ where $\boldsymbol{\phi} : \mathbb{R}^{d_h} \to \mathbb{R}^{d_h}$ is applied element-wise:

$$\text{LLA}(\mathbf{Q}, \mathbf{K}, \mathbf{V})_t \;=\; \frac{\boldsymbol{\phi}(\mathbf{q}_t)^\top \big(\sum_{s=1}^{n} \boldsymbol{\phi}(\mathbf{k}_s)\mathbf{v}_s^\top\big)}{\boldsymbol{\phi}(\mathbf{q}_t)^\top \big(\sum_{s=1}^{n} \boldsymbol{\phi}(\mathbf{k}_s)\big)}. \tag{8}$$

The context matrix $\mathbf{M} = \sum_s \boldsymbol{\phi}(\mathbf{k}_s)\mathbf{v}_s^\top \in \mathbb{F}^{d_h \times d_h}$ depends only on the keys/values and is computed once: per-token output costs $O(d_h^2)$ and the per-block prover cost becomes $O(nd_h^2)$ instead of $O(n^2 d_h)$. The denominator is computed for training stability [6]; the SNARK currently proves the un-normalised numerator (§8).

Additive sub-table feature map. Rather than a fixed kernel, π-Former parametrises $\boldsymbol{\phi}$ as a sum of learnable sub-table lookups. Let b be the input bit-width and $c \mid b$ the decomposition depth, $\beta = b/c$ bits per chunk. For scalar x, define the integer index $x_{\text{int}} = \text{clamp}(\lfloor x/s \rfloor, 0, 2^b - 1)$ and chunk extractor $\chi_i(x_{\text{int}}) = (x_{\text{int}} \gg i\beta) \,\&\, (2^\beta - 1)$. Then

$$\boldsymbol{\phi}(x) \;=\; \sum_{i=0}^{c-1} T_i\big[\chi_i(x_{\text{int}})\big], \tag{9}$$

where $T_0, \ldots, T_{c-1} \in \mathbb{R}^{2^\beta}$ are learnable sub-tables, initialised so $\sum_i T_i \approx \text{GELU}$ and trained end-to-end with the straight-through estimator [4]. For $b = 16, c = 2$ this replaces a 65,536-entry monolithic table with two 256-entry tables: *exactly* the Lasso structured condition.

3.2 Ternary projections

All projection matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O, \mathbf{W}_1, \mathbf{W}_2$ and the LM head are constrained to entries in $\mathcal{T} = \{-1, 0, +1\}$ scaled by a single learnable scalar $\alpha \in \mathbb{F}$ per layer, following the BitNet b1.58 design [9]. Forward pass: $\mathbf{y} = \alpha \cdot (\mathbf{x}\mathbf{W}) + \mathbf{b}$ with $\mathbf{W} \in \mathcal{T}^{m \times n}$. Training uses the magnitude-thresholded mapping $w \mapsto \text{sign}(w) \cdot [\,|w| > 0.7\overline{|w|}\,]$ with STE. The scalar α is multiplied in *after* the matrix product so the in-circuit weight stays ternary.

In the SNARK, a sumcheck for a ternary contraction $\mathbf{Y}(r_t, r_d) - \mathbf{b}(r_d) = \alpha \sum_k \mathbf{X}(r_t, k)\mathbf{W}(k, r_d)$ materialises only the cheap vector $\alpha \cdot \mathbf{W}(\cdot, r_d)$, never a field weight matrix; the prover folds the equality polynomial against ternary entries with $+=$/$-=$/skip rather than field multiplications. A stand-alone Lasso check against the 4-element table $[0, 1, r-1, 0]$ enforces ternarity and is a candidate to fold into setup-time preprocessing, so the verifying key carries a binding proof that $\mathbf{W} \in \mathcal{T}^{m \times n}$.

3.3 Constraint-fused LayerNorm

Let $\mathbf{x} \in \mathbb{Z}^d$ be a row, $\mathrm{sum_x} = \sum_j x_j$, $\mathrm{var_x} = \sum_j (d \cdot x_j - \mathrm{sum_x})^2$ the integer numerator, and $\sigma = \lfloor \sqrt{\mathrm{var_x}}/d \rfloor$. π-Former proves the entire LayerNorm at a single random row index $r_t \in \mathbb{F}^{t_{\mathrm{bits}}}$ in four steps, with no field divisions and no square roots:

1. **Mean sumcheck.** Degree-2 sumcheck for $\widetilde{\mathrm{sum_x}}(r_t) = \sum_j x_{\mathrm{col}}(j)$ where $x_{\mathrm{col}} = \widetilde{\mathbf{x}}.\mathrm{fix_row}(r_t)$.
2. **Variance sumcheck.** Cubic sumcheck for $\widetilde{\mathrm{var_x}}(r_t) = \sum_j h(j)^2$ with $h(j) = d \cdot x_{\mathrm{col}}(j) - \widetilde{\mathrm{sum_x}}(r_t)$.
3. **Sigma range proof.** Show σ is the integer floor square-root by range-checking $\mathrm{var_x} - (d\sigma)^2 \geq 0$ and $(d\sigma + d)^2 - 1 - \mathrm{var_x} \geq 0$ through the chunked Lasso range proof.
4. **Y constraint fusion.** For a single random (r_{y_t}, r_{y_d}), audit $\boldsymbol{\gamma} \odot (d\mathbf{x} - \mathrm{sum_x}) + \boldsymbol{\beta} \cdot d\sigma \approx d\sigma \cdot \mathbf{y}$ (up to integer rounding) by another range proof on the lo/hi residuals; the $\gamma\mathbf{X}$ and $\sigma\mathbf{Y}$ legs are fused into a single multi-batched cubic sumcheck.

The verifier evaluates $\boldsymbol{\gamma}$ and $\boldsymbol{\beta}$ directly from the VK in $O(d)$ field operations per layer; everything else is one shared opening per witness.

4 The π-Former Protocol

This section gives the complete protocol. Algorithm 1 describes preprocessing; Algorithm 2 the model-level prover; Algorithm 3 the per-block "Phase 1" that commits intermediate matrices and proves both LayerNorms; and Algorithm 7 the verifier. The cross-block batched sumcheck primitive is given separately as Algorithm 4, the LayerNorm prover as Algorithm 5, and the global batched range proof as Algorithm 6.

4.1 Preprocessing

Algorithm 1 π-Former. $\mathsf{Setup}(\theta,\ \mathrm{params})$

Input: model weights $\theta = \{\mathbf{W}_Q^\ell, \mathbf{W}_K^\ell, \mathbf{W}_V^\ell, \mathbf{W}_O^\ell, \mathbf{W}_1^\ell, \mathbf{W}_2^\ell, \boldsymbol{\gamma}^\ell, \boldsymbol{\beta}^\ell\}_{\ell=1}^L$, final-LN params, LM head $\mathbf{W}_h$; params $(T, d, d_{ff}, d_h, b, c)$.
Output: proving key pk, verifying key vk.

```
derive Hyrax generators g_j ← SHA3("piformer-hyrax-gen"‖j) · G for j = 0, ..., √N.
for each weight matrices W ∈ θ ∪ {W_h} do
    C_W ← Com(W)                                          ▷ Hyrax commitment
end for
for each LLA sub-tables T_0, ..., T_{c−1} do
    C_{T_i} ← Com(T_i)
end for
vk ← {C_W} ∪ {C_{T_i}} ∪ {γ^ℓ, β^ℓ} ∪ {g_j}
pk ← vk ∪ {raw ternary arrays}
return (pk, vk)
```

The verifying key contains *no* raw weights, only commitments; the proving key additionally carries the ternary $\{-1, 0, +1\}$-valued arrays so the prover can fold the equality polynomial against them in $O(Td)$ time without any field multiplications.

4.2 The model-level prover

The key structural choices are: *(i) one global random point r_{td} is drawn after all per-block commitments are absorbed*, so all six cross-block sumchecks can share it; *(ii) ten cross-cutting Hyrax accumulators* (LayerNorm row / td, projection W/b, LM-head W/b, range σ/Y/m, intermediate) are pushed by every claim and finalised once at the end of the *verifier*, with one MSM pair per group; *(iii) residuals are verified by linearity of the Hyrax PCS:* $C_{\mathbf{X}_{\mathrm{mid}}} = C_{\mathbf{X}_{\mathrm{in}}} + C_{\mathbf{O}_{\mathrm{attn}}}$ is computed by the verifier from the existing commitments and requires no proof.

Algorithm 2 π-Former. $\mathsf{Prove}(\mathsf{pk}, W, \mathsf{inst})$

Input: proving key pk, witness $W = \{\mathbf{x}_{\text{in}}, (\text{block witnesses})_{\ell=1}^{L}, \text{final-LN}, \text{LM-head}\}$, instances $\mathsf{inst} = \{(\boldsymbol{\phi}(\mathbf{Q}), \boldsymbol{\phi}(\mathbf{K}), \boldsymbol{\phi}(\mathbf{A}))_\ell\}$.
Output: model proof π.

$\mathsf{FS.init}(\texttt{"piformer"})$; $C_{\mathbf{x}_{\text{in}}} \leftarrow \mathsf{Com}(\mathbf{x}_{\text{in}})$; $\mathsf{FS.absorb}(C_{\mathbf{x}_{\text{in}}})$
Phase 1: per-block commit + LayerNorm
$C_{\text{cur}} \leftarrow C_{\mathbf{x}_{\text{in}}}$
for $\ell = 1, \ldots, L$ **do**
  $p_\ell \leftarrow \textsc{Phase1}(W_\ell, C_{\text{cur}}, \mathsf{pk}_\ell, \mathsf{FS})$ ▷ Algorithm 3
  $C_{\text{cur}} \leftarrow p_\ell.C_{\mathbf{x}_{\text{mid}}} + p_\ell.C_{\mathbf{O}_{\text{ffn}}}$ ▷ homomorphic residual
end for
Phase 2: global random point
$r_{td} = (r_t \| r_{\text{out}}) \leftarrow \mathsf{FS.challenge}(t_{\text{bits}} + d_{\text{bits}})$
Phase 3: six cross-block batched sumchecks
$\pi_{qkv} \leftarrow \textsc{CrossBatch}(\{f_\ell^{qkv}, g_\ell^{qkv}\}_\ell, \mathsf{FS})$
$\pi_{op} \leftarrow \textsc{CrossBatch}(\{f_\ell^{op}, g_\ell^{op}\}_\ell, \mathsf{FS})$
$\pi_{ao} \leftarrow \textsc{CrossBatch}(\{f_\ell^{ao}, g_\ell^{ao}\}_\ell, \mathsf{FS})$
$\pi_{ac} \leftarrow \textsc{CrossBatch}(\{f_\ell^{ac}, g_\ell^{ac}\}_\ell, \mathsf{FS})$
▷ CrossBatch defined in Algorithm 4
Phase 4: per-block FFN Lasso, then cross-block FFN sumchecks
for $\ell = 1, \ldots, L$ **do**
  $\pi_{\text{lasso},\ell}^{\text{ffn}} \leftarrow \mathsf{Prove}\text{-Lasso}(\mathsf{inst}_\ell.\text{ffn}, \mathsf{pk}_\ell, \mathsf{FS})$
end for
$\pi_{ffy} \leftarrow \textsc{CrossBatch}(\{f_\ell^{ffy}, g_\ell^{ffy}\}_\ell, \mathsf{FS})$
$\pi_{ffm} \leftarrow \textsc{CrossBatch}(\{f_\ell^{ffm}, g_\ell^{ffm}\}_\ell, \mathsf{FS})$
Phase 5: final LayerNorm + LM head
$\pi_{\text{final}} \leftarrow \textsc{ProveLN}(W.\text{final-LN}, \mathsf{pk}.\text{final-LN}, \mathsf{FS})$
$\pi_{\text{head}} \leftarrow \mathsf{Prove}\text{-Proj}(W.\text{LM-head}, \mathsf{pk}.\text{LM-head}, \mathsf{FS})$
Phase 6: global intermediate batch open
$\pi_{\text{inter}} \leftarrow \mathsf{Open}\text{-Batch}\big(\{\mathbf{Q}_\ell, \mathbf{K}_\ell, \mathbf{V}_\ell, \mathbf{O}_{\text{attn}}^\ell, \mathbf{O}_{\text{ffn}}^\ell\}_{\ell=1}^{L},\ r_{td}\big)$
Phase 7: 17 cross-block matrix-type batch opens
$\{\pi_{\text{open}}^{(j)}\}_{j=1}^{17} \leftarrow \mathsf{Open}\text{-Batch}\big(\{\mathbf{X}_{\text{n1}}^\ell, \mathbf{W}_Q^\ell, \ldots\}_\ell,\ r^{(j)}\big)$
Phase 8: global Lasso
$\pi_{\text{lasso}} \leftarrow \mathsf{Prove}\text{-LassoMulti}\big(\{\mathsf{inst}_\ell.\boldsymbol{\phi}(\mathbf{Q}), \mathsf{inst}_\ell.\boldsymbol{\phi}(\mathbf{K})\}_\ell, \mathsf{FS}\big)$
return $\pi = \{C_{\mathbf{x}_{\text{in}}}, (p_\ell)_\ell, \pi_{qkv}, \pi_{op}, \pi_{ao}, \pi_{ac}, \pi_{ffy}, \pi_{ffm}, \pi_{\text{final}}, \pi_{\text{head}}, \pi_{\text{inter}}, \{\pi_{\text{open}}^{(j)}\}, \pi_{\text{lasso}}\}$

Algorithm 3 $\textsc{Phase1}(W_\ell, C_{\text{cur}}, \mathsf{pk}_\ell, \mathsf{FS})$

Input: block witness, current input commitment, block proving key.
Output: intermediate commitments + LN1/LN2 sub-proofs + block range m.

commit 7 intermediate matrices: $C_{\mathbf{X}_{\text{n1}}}, C_{\mathbf{Q}}, C_{\mathbf{K}}, C_{\mathbf{V}}, C_{\mathbf{O}_{\text{attn}}}, C_{\mathbf{X}_{\text{n2}}}, C_{\mathbf{O}_{\text{ffn}}}$
// Global range batch for both LayerNorms in this block (4 witnesses share m)
$\mathsf{rw}_1 \leftarrow \textsc{ComputeRangeWitnesses}(W_\ell.\text{LN1})$
$\mathsf{rw}_2 \leftarrow \textsc{ComputeRangeWitnesses}(W_\ell.\text{LN2})$
$(\mathsf{rps}, m^\ell, \mathsf{rvs}) \leftarrow \textsc{ProveRangeBatched}([\mathsf{rw}_1.\sigma, \mathsf{rw}_1.y, \mathsf{rw}_2.\sigma, \mathsf{rw}_2.y], 32, \mathsf{FS})$ ▷ Algorithm 6
$\pi_{\text{LN1}} \leftarrow \textsc{ProveLN}(W_\ell.\text{LN1}, \{C_{\text{cur}}, C_{\mathbf{X}_{\text{n1}}}\}, \mathsf{rps}_{0,1}, \mathsf{FS})$
$\mathsf{FS.absorb}(C_{\mathbf{Q}}, C_{\mathbf{K}}, C_{\mathbf{V}}, C_{\mathbf{O}_{\text{attn}}})$
$C_{\mathbf{X}_{\text{mid}}} \leftarrow C_{\text{cur}} + C_{\mathbf{O}_{\text{attn}}}$ ▷ homomorphic residual 1
$\pi_{\text{LN2}} \leftarrow \textsc{ProveLN}(W_\ell.\text{LN2}, \{C_{\mathbf{X}_{\text{mid}}}, C_{\mathbf{X}_{\text{n2}}}\}, \mathsf{rps}_{2,3}, \mathsf{FS})$
$\mathsf{FS.absorb}(C_{\mathbf{O}_{\text{ffn}}})$
return $\{C_{\mathbf{X}_{\text{n1}}}, C_{\mathbf{Q}}, C_{\mathbf{K}}, C_{\mathbf{V}}, C_{\mathbf{O}_{\text{attn}}}, C_{\mathbf{X}_{\text{n2}}}, C_{\mathbf{O}_{\text{ffn}}}, C_{\mathbf{X}_{\text{mid}}}, \pi_{\text{LN1}}, \pi_{\text{LN2}}, m^\ell\}$

4.3 Per-block Phase 1

The only truly per-block step is Phase 1: it commits seven intermediate matrices, runs the global range batch for both LayerNorms, and proves LN1/LN2. The remaining sub-protocols (QKV, output projection, attention, FFN) are *not* run independently per block; they are batched cross-block in Phase 3 of Algorithm 2.

4.4 Cross-block batched sumcheck

Algorithm 4 CROSSBATCH($\{f_\ell, g_\ell\}_{\ell=1}^{L}$, FS)

Input: per-block multilinear polynomials $f_\ell, g_\ell : \mathbb{F}^n \to \mathbb{F}$ and per-block targets H_ℓ.
Output: batched sumcheck proof π_{batch} + shared random point $r \in \mathbb{F}^n$.

for $\ell = 1, \ldots, L$ **do**
  absorb per-block claim H_ℓ and any per-block constants into FS
end for
$\eta \leftarrow$ FS.challenge("batch_eta")
$H^\star \leftarrow \sum_{\ell=1}^{L} \eta^{\ell-1} H_\ell$
$P(x) \leftarrow \sum_{\ell=1}^{L} \eta^{\ell-1} f_\ell(x) g_\ell(x)$
for $i = 1, \ldots, n$ **do**
  send univariate $s_i(X) = \sum_{x \in \{0,1\}^{n-i}} P_i(X, x)$ at $X \in \{0, 1, 2\}$
  $r_i \leftarrow$ FS.challenge("sumcheck_r"$\|i$)
end for
return $\pi_{\text{batch}} = \{\eta, (s_i)_{i=1}^{n}\}$, $r = (r_1, \ldots, r_n)$

The verifier recovers $H^\star$ from the per-block claims and η, then runs the standard sumcheck verifier on π_{batch} to a final point r. From r the verifier checks the terminal claim $P(r) = \sum_{\ell=1}^{L} \eta^{\ell-1} f_\ell(r) g_\ell(r)$ against the per-block evaluations opened later via the cross-block batch opens. By Theorem 5.3 below, this protocol replaces L independent sumcheck transcripts with one of length n at a soundness cost of an additive $(L-1)/|\mathbb{F}|$.

4.5 Constraint-fused LayerNorm prover

Algorithm 5 PROVELN($W, \{C_{\mathbf{x}}, C_{\mathbf{y}}\}, \mathsf{vk}_{\text{LN}}, (\mathsf{rps}_\sigma, \mathsf{rps}_y)$, FS)

Input: witness $W = \{\mathbf{x}, \mathbf{y}, \text{sum_x}, \text{sq_sum_x}, \sigma\}$, IO commitments, range witnesses (precomputed).
Output: LN proof; advances Hyrax accumulators $\mathsf{acc}_t, \mathsf{acc}_{td}$.

commit $C_{\text{sum_x}}, C_{\text{sq_sum}}, C_\sigma$; absorb everything into FS
$r_t \leftarrow$ FS.challenge(t_{bits})
// 1. Mean sumcheck
$(\pi^{\text{mean}}, r_d^{\text{mean}}) \leftarrow$ Prove-Sumcheck$_2(\widetilde{\mathbf{x}}_{\text{col}}, \widetilde{\mathbb{1}}, \widetilde{\text{sum_x}}(r_t)$, FS)
// 2. Variance sumcheck (cubic)
$h(j) \leftarrow d \cdot \widetilde{\mathbf{x}}_{\text{col}}(j) - \widetilde{\text{sum_x}}(r_t)$
$(\pi^{\text{var}}, r_d^{\text{var}}) \leftarrow$ Prove-Sumcheck$_3(h, h, h, \widetilde{\text{var_x}}(r_t)$, FS)
// 3. Sigma range proof: var$-(d\sigma)^2 \geq 0$ *and* $(d\sigma + d)^2 - 1 -$*var* ≥ 0
$r_\sigma \leftarrow \mathsf{rps}_\sigma.r$; **check** sigma constraint algebraically at r_σ
// 4. Y constraint fusion: $\gamma\mathbf{X} + \sigma\mathbf{Y}$ *legs into one multi-batched cubic sumcheck*
$r_y \leftarrow \mathsf{rps}_y.r$; $\alpha \leftarrow$ FS.challenge("layernorm_alpha")
$(\pi^{\gamma\sigma}, r_f) \leftarrow$ Prove-Sumcheck$_3^{\text{multi}}\big([\boldsymbol{\gamma} \odot \widetilde{\mathbf{x}}_{\text{ry}}, \widetilde{\sigma}_{\text{ry}} \odot \widetilde{\mathbf{y}}_{\text{ry}}], \alpha$, FS$\big)$
push 10 Hyrax claims on $C_{\text{sum_x}}, C_{\text{sq_sum}}, C_\sigma, C_{\mathbf{x}}, C_{\mathbf{y}}$ into $\mathsf{acc}_t, \mathsf{acc}_{td}$
return $\{\pi^{\text{mean}}, \pi^{\text{var}}, \pi^{\gamma\sigma}$, evaluations$\}$

4.6 Globally batched range proof

The key win is the shared m_{com}: instead of B independent multiplicity commitments ($B \times \sqrt{2^{16}}$ MSMs), there is exactly one. For $B = 4$ (two LayerNorms per block, each with σ and y witnesses) this saves $3 \times \sqrt{2^{16}} = 768$ MSMs per block.

Algorithm 6 PROVERANGEBATCHED($[V^{(b)}]_{b=1}^{B}, b{=}32, \mathsf{FS}$)

Input: B witnesses $V^{(b)} \in \mathbb{F}^{2^{n_b}}$; bit-width b (CHUNK_BITS=16).
Output: per-witness proofs rps_b, shared multiplicity proof, evaluation points.

Phase 1: commit chunks
for $b = 1, \ldots, B$ **do**
  $V_{lo}^{(b)}[i] \leftarrow V^{(b)}[i] \bmod 2^{16}$; $V_{hi}^{(b)}[i] \leftarrow V^{(b)}[i] \gg 16$
  $\mathsf{cc}_0^{(b)} \leftarrow \mathsf{Com}(V_{lo}^{(b)})$; $\mathsf{cc}_1^{(b)} \leftarrow \mathsf{Com}(V_{hi}^{(b)})$; absorb both
end for
Phase 1b: shared multiplicity commitment
$m[v] \leftarrow$ count of v across all chunks of all $V^{(b)}$, $v \in [0, 2^{16})$
$m_{\mathrm{com}} \leftarrow \mathsf{Com}(m)$; $\mathsf{FS.absorb}(m_{\mathrm{com}})$
Phase 2: per-witness sumcheck + chunk batch open
for $b = 1, \ldots, B$ **do**
  claim $\sum_i V^{(b)}[i]$; run sumcheck to point $r_v^{(b)}$
  check $V^{(b)}(r_v^{(b)}) = \mathsf{cc}_0^{(b)}(r_v^{(b)}) + 2^{16}\mathsf{cc}_1^{(b)}(r_v^{(b)})$
  HYRAXOPENBATCH($[V_{lo}^{(b)}, V_{hi}^{(b)}], r_v^{(b)}$)
end for
Phase 3: shared multiplicity opening (LogUp)
$r_m \leftarrow \mathsf{FS.challenge}(16)$; $\mathsf{Open}(m, r_m)$
return $(\mathsf{rps}_{1,\ldots,B}, (m_{\mathrm{com}}, m(r_m), \mathsf{Open}), (r_v^{(b)})_b)$

4.7 The verifier

Algorithm 7 π-Former. $\mathsf{Verify}(\mathsf{vk}, \pi, \mathsf{inst}, \mathbf{x}_{\mathrm{in}}, \mathbf{y})$

check $\mathsf{Com}(\mathbf{x}_{\mathrm{in}}) = \pi.C_{\mathbf{x}_{\mathrm{in}}}$ and $\mathsf{Com}(\mathbf{y}) = \pi.C_{\mathbf{y}}$
$\mathsf{FS.init}$(`"piformer"`); absorb $\pi.C_{\mathbf{x}_{\mathrm{in}}}$
initialise 10 deferred Hyrax accumulators
$C_{\mathrm{cur}} \leftarrow \pi.C_{\mathbf{x}_{\mathrm{in}}}$
for $\ell = 1, \ldots, L$ **do** ▷ Phase 1 lockstep with prover
  VERIFYRANGEBATCHED(p_ℓ.range, $\ldots$)
  VERIFYLN($p_\ell.\pi_{\mathrm{LN1}}, \{C_{\mathrm{cur}}, p_\ell.C_{\mathbf{X}_{\mathrm{n1}}}\}, \ldots$)
  absorb $p_\ell.C_{\mathbf{Q}}, C_{\mathbf{K}}, C_{\mathbf{V}}, C_{\mathbf{O}_{\mathrm{attn}}}$
  $C_{\mathbf{X}_{\mathrm{mid}}} \leftarrow C_{\mathrm{cur}} + p_\ell.C_{\mathbf{O}_{\mathrm{attn}}}$
  VERIFYLN($p_\ell.\pi_{\mathrm{LN2}}, \{C_{\mathbf{X}_{\mathrm{mid}}}, p_\ell.C_{\mathbf{X}_{\mathrm{n2}}}\}, \ldots$)
  absorb $p_\ell.C_{\mathbf{O}_{\mathrm{ffn}}}$
  $C_{\mathrm{cur}} \leftarrow C_{\mathbf{X}_{\mathrm{mid}}} + p_\ell.C_{\mathbf{O}_{\mathrm{ffn}}}$
end for
$r_{td} \leftarrow \mathsf{FS.challenge}(t_{\mathrm{bits}} + d_{\mathrm{bits}})$
verify six cross-block sumchecks $\pi_{qkv}, \pi_{op}, \pi_{ao}, \pi_{ac}, \pi_{ffy}, \pi_{ffm}$
verify final LayerNorm + LM-head proofs
verify global intermediate batch open at r_{td}
verify 17 cross-block matrix-type batch opens
FINALISE($\mathsf{acc}_t, \mathsf{acc}_{td}, \ldots$) ▷ 10× one MSM pair
verify global Lasso π_{lasso}
return ACCEPT

5 Soundness

We sketch the soundness analysis; the full proofs follow standard Schwartz–Zippel and PCS-binding arguments and are given in the extended version.

Lemma 5.1 (Sumcheck soundness)**.** *For an MLE f of individual degree $\leq \delta$, a cheating prover can make the sumcheck for $H = \sum_{x \in \{0,1\}^n} f(x)$ accept a false claim $H' \neq H$ with probability at most $n\delta/|\mathbb{F}|$.*

Lemma 5.2 (Hyrax binding)**.** *Under the discrete-log assumption on $\mathbb{G}$, the Hyrax PCS is computationally binding: no PPT adversary can open a single commitment to two different evaluations at the same point with non-negligible advantage.*

Theorem 5.3 (Cross-block batching is sound)**.** *Let* CROSSBATCH *(Algorithm 4) reduce L per-block claims $\{H_\ell\}_{\ell=1}^{L}$ to one batched claim $H^\star = \sum_\ell \eta^{\ell-1} H_\ell$, and let the inner sumcheck be sound with error $\leq n\delta/|\mathbb{F}|$ (Lemma 5.1). Then* CROSSBATCH *is sound with total error*

$$\varepsilon_{\text{batch}} \;\leq\; \frac{L-1}{|\mathbb{F}|} + \frac{n\delta}{|\mathbb{F}|}.$$

Proof. The challenge η is drawn from FS *after* every per-block claim and per-block witness commitment is absorbed. Therefore the prover is committed to $\{H_\ell\}_{\ell=1}^{L}$ before learning η. If any single claim is false, $\hat{H}_{\ell^\star} \neq H_{\ell^\star}$, then $P(\eta) = \sum_\ell \eta^{\ell-1}(\hat{H}_\ell - H_\ell)$ is a non-zero polynomial in η of degree $\leq L-1$. By Schwartz–Zippel, $P(\eta) = 0$ holds for at most $L-1$ values of $\eta \in \mathbb{F}$, so a uniformly random η catches the false claim with probability $\geq 1 - (L-1)/|\mathbb{F}|$. Conditional on the combined claim being honest, the inner sumcheck has soundness $n\delta/|\mathbb{F}|$. A union bound gives the stated error. □

Theorem 5.4 (Homomorphic residuals)**.** *Let $C_{\mathbf{A}} = \mathsf{Com}(\mathbf{A})$, $C_{\mathbf{B}} = \mathsf{Com}(\mathbf{B})$ for committed matrices $\mathbf{A}, \mathbf{B}$. The verifier-computed commitment $C_{\mathbf{A}} + C_{\mathbf{B}}$ binds $\mathbf{A} + \mathbf{B}$ unconditionally given Hyrax binding (Lemma 5.2).*

Proof. Hyrax commits row-by-row as MSMs over public generators $\mathbf{g}$. By bilinearity of MSM, $C_{\mathbf{A}} + C_{\mathbf{B}} = (\mathrm{MSM}(\mathbf{g}, A_i) + \mathrm{MSM}(\mathbf{g}, B_i))_i = (\mathrm{MSM}(\mathbf{g}, A_i + B_i))_i = \mathsf{Com}(\mathbf{A} + \mathbf{B})$. Binding of $\mathbf{A} + \mathbf{B}$ then follows from binding of $\mathbf{A}$ and $\mathbf{B}$. □

Theorem 5.5 (π-Former end-to-end soundness)**.** *For any PPT adversary $\mathcal{A}$, the probability that $\mathsf{Verify}(\mathsf{vk}, \pi, \mathsf{inst}, \mathbf{x}_{\text{in}}, \mathbf{y}) = \mathsf{ACCEPT}$ but $\mathcal{M}_\theta(\mathbf{x}_{\text{in}}) \neq \mathbf{y}$ is at most*

$$\varepsilon_{\text{sound}} \;\leq\; \frac{m_{\text{sc}}\, n_{\max}\delta_{\max} + m_{\text{batch}}(L-1) + m_{\text{open}}}{|\mathbb{F}|} + \varepsilon_{\text{DL}},$$

where m_{sc} is the number of sumcheck invocations after batching, $m_{\text{batch}} = 6$ is the number of cross-block batches, m_{open} covers the random-linear-combination checks used in batched openings/lookups, $n_{\max}$ is the maximum number of sumcheck variables, $\delta_{\max} \leq 3$ is the maximum round-polynomial degree, and ε_{DL} is the DL advantage of $\mathcal{A}$ on $\mathbb{G}$.

Proof sketch. We proceed by hybrid argument over the L blocks. (i) By Lemma 5.2, any forged Hyrax opening yields a DL solution. (ii) Block-level chaining is sound by Theorem 5.4: $C^{\ell}_{\mathbf{X}_{\text{out}}}$ is the homomorphic sum of its constituent committed tensors, so block ℓ's output binds inductively from the public input $\mathbf{x}_{\text{in}}$. (iii) Within each block, the global range batch (Algorithm 6) is sound because the shared multiplicity m_{com} is committed before any per-witness sumcheck challenge is drawn (LogUp cannot be satisfied for an out-of-range chunk value because the identity table $T_{\text{id}}[i] = i$ has no entry for $v \geq 2^{16}$); LayerNorm is sound by composition of the mean, variance, sigma-range, and Y-fusion sub-claims; the six cross-block sumchecks are sound by Theorem 5.3; and Lasso is sound by Lemma 5.2 plus Lemma 5.1. (iv) The final layer + LM head reduce to one block. (v) The global Lasso π_{lasso} is sound by Lemma 5.2 per sub-table with batch error bounded by the number of sub-tables times the per-table error. A union bound over the batched sumchecks, batched openings, lookup arguments, and LayerNorm sub-proofs gives the stated bound. For typical parameters ($L \leq 12$, $n_{\max} \leq 64$, $\delta_{\max} \leq 3$) over BN254, the statistical term remains below 2^{-240} for the implemented protocol counts and is dominated by the DL term. □

5.1 Zero knowledge

The current implementation is *not* zero-knowledge: weight commitments are public VK entries, and intermediate evaluation claims (e.g., $\mathbf{X}_{\text{n1}}(r_t, r_d)$) are revealed in the sumcheck transcripts. Standard techniques apply: blinding all witness polynomials with random masking terms before commitment, and using zero-knowledge variants of sumcheck and Hyrax. We discuss this further in §8.

6 Implementation

We implement π-Former in $\sim$14 k lines of Rust over BN254 plus a PyTorch training pipeline. The Rust prover/verifier and CLI are organised as in Table 1.

Table 1: π-Former Rust implementation, lines of code per module.

Module	**LoC**
`poly/` (dense MLE, equality, helpers)	∼700
`pcs/` (Hyrax PCS, batch accumulator)	∼700
`subprotocols/` (sumcheck ×3, GKR combine)	∼1,450
`lookup/` (Lasso multi, range batched)	∼1,300
`attention/` (LN, projection, LLA, ternary check)	∼3,200
`ffn/` (FFN sub-prover)	∼700
`cross_layer/` (cross-layer projection sumcheck)	∼500
`prover.rs`, `verifier.rs`, `setup.rs`	∼3,900
`bin/piformer/` (CLI, codecs, sample)	∼1,500
Total	∼14,000

Table 2: End-to-end π-Former performance on three model sizes (single-thread, BN254). Times are pure compute; IO excluded. "vk" includes all weight + sub-table commitments.

Model	d	L	d_{ff}	T	**Params**	**Setup**	**Prove**	**Verify**	**Proof**	**vk**
tiny	8	1	16	8	1.1 k	0.02 s	0.22 s	0.03 s	108 KB	5.7 KB
small	64	2	256	16	133 k	0.09 s	1.10 s	0.15 s	1.0 MB	62 KB
medium	128	4	512	32	924 k	0.29 s	4.24 s	0.28 s	5.5 MB	219 KB

PyTorch pipeline. The training pipeline implements `TernaryLinear` (with STE), `StructuredLookupActivation` (ϕ with sub-table forward + STE backward), and `LinearAttentionLayer`. The integer forward pass used to construct the witness is bit-identical to the Rust circuit, so a witness exported from PyTorch is accepted by the Rust prover with no quantisation gap.

Single-head, no causal mask, no ZK. The current Rust prover supports single-head attention with no causal masking and no zero-knowledge masking layer; multi-head splits the attention sumchecks into parallel batches with the same protocol structure, and ZK is an additional blinding layer over committed witnesses (§8).

7 Evaluation

Setup. Times are wall-clock on a single x86_64 CPU core (single-threaded), measured by the Rust CLI itself: setup, prove, and verify times exclude JSON parsing and proving/verifying-key file IO; these are the inner timings reported by the prover/verifier functions. Proof sizes are the on-disk size of the binary proof bundle (proof + public lookup outputs + version header).

7.1 End-to-end performance

Table 2 shows end-to-end performance on three model sizes. Prove time scales roughly linearly in $L \cdot T \cdot d$ and $L \cdot T \cdot d_{ff}$ (the per-block sumchecks are $O(Td)$/$O(Td_{ff})$; cross-block batching collapses transcript work but not arithmetic work). Verifier compute is sub-linear in the model size because every cross-block sumcheck is a $\log Td$-round transcript and the deferred Hyrax accumulators finalise in one MSM pair per group.

7.2 Verifier per-phase breakdown

Table 3 breaks down the instrumented verifier phases. Three observations: (i) cross-block sumcheck transcript checks are negligible (<1 ms total) regardless of model size, validating that the batching collapses verifier transcript work; (ii) the `cross_batch_opens` and `acc_finalize` phases (Hyrax opening verification and accumulator finalisation) dominate the instrumented compute, scaling with the size of the committed matrices; and (iii) the global Lasso verification is sub-linear in the model size as expected (one combined sumcheck plus one Hyrax opening).

Table 3: Verifier compute breakdown by phase (ms). All values are single-thread wall-clock from the Rust CLI's internal phase timers; the subtotal omits parser, transcript setup, and top-level dispatch overheads included in Table 2.

Phase	**tiny**	**small**	**medium**
`batch_qkv` (cross-block QKV)	0.03	0.12	0.37
`batch_oproj` (output projection)	0.02	0.05	0.13
`batch_attn` (attention out + ctx)	0.03	0.07	0.15
`batch_ffn_y`	0.02	0.11	0.04
`batch_ffn_m`	0.02	0.06	0.03
`final_range`	1.52	1.49	3.13
`final_ln`	0.14	0.23	0.64
`lm_head`	0.04	0.19	0.13
`inter_batch` (5L matrices, one open)	1.77	2.86	1.20
`cross_batch_opens` (17 batched opens)	3.58	44.53	46.58
`lasso` (global Lasso)	1.57	25.58	40.93
`acc_finalize` (10 deferred MSMs)	13.20	23.75	54.15
Instrumented subtotal	22	99	148

Table 4: Qualitative comparison with prior verifiable-inference systems for transformers. All systems target a non-interactive SNARK.

System	**Attention**	**Activation**	**PCS**	**Setup**
zkLLM [13]	softmax (`tlookup`)	structured lookup	custom IOP	transparent
zkGPT [10]	softmax (approx.)	lookup table	Plonky2	transparent
π-Former (this work)	**linear (LLA, exact)**	learned structured lookup	Hyrax	transparent

7.3 Comparison with prior work

Table 4 contrasts π-Former with prior systems qualitatively. Two structural differences are worth highlighting. First, π-Former's attention is not an approximation of softmax: it is a different (learnable, kernelised) attention mechanism that is trained from scratch and is therefore an *exact*, not an approximate, computation. Second, the activation ϕ is a *learnable* additive lookup, so the same protocol path that proves ϕ also lets training adapt the activation to the data. A full quantitative comparison on matched architectures and quantisation settings is left to future work.

8 Discussion and Limitations

What is and is not in the SNARK. The Rust prover proves the un-normalised LLA output $\phi(\mathbf{Q})(\phi(\mathbf{K})^\top \mathbf{V})\mathbf{W}_O$. The Python forward pass adds the row-wise normaliser $Z_t = \phi(\mathbf{q}_t)^\top \sum_s \phi(\mathbf{k}_s)$ for training stability; matching the two requires either disabling Z at export time or extending the circuit with a row-normalisation sumcheck. The current implementation also lacks causal masking, multi-head attention, and zero-knowledge masking. None of these changes the protocol architecture: causal masking is a sumcheck-time indicator polynomial, multi-head splits the attention sumchecks into parallel batches, and ZK is an additional blinding layer over committed witnesses.

Model-quality evaluation. The evaluation in §7 measures proof-system performance and end-to-end arithmetic consistency; it is not yet an ML benchmark claim. Because π-Former changes the attention kernel and constrains weights to ternary values, a deployment-quality comparison must train matched baselines under the same parameter budget, context length, dataset, and quantisation regime, then report both task quality and proof cost. The present results should therefore be read as evidence that the co-designed architecture is efficiently provable, not as evidence that it preserves the accuracy of a softmax transformer at scale.

From prototype to deployment. Our timings are CPU and single-threaded. GPU MSM and parallel sumcheck [10] would translate directly to π-Former: every dominant inner loop (cross-block batched sumcheck, Lasso multi- instance, batched Hyrax open) is data-parallel, and the global Hyrax accumulators are pure MSMs at the boundary. Streaming token generation is the natural fit for Nova-style folding [7]: the running context matrix $\mathbf{M} = \phi(\mathbf{K})^\top \mathbf{V}$ is the IVC accumulator, and one folding step per token yields constant amortised proving cost.

Beyond π-Former. The Rust codebase already contains two stand-alone sub-protocols that are not yet wired into the end-to-end prover. The first is an in-circuit ternary-weight check (`attention/ternary_check.rs`) via a 4-element

Lasso table $[0, 1, r-1, 0]$; folding this into setup yields a verifying key that carries a binding proof $\mathbf{W} \in \mathcal{T}^{m \times n}$. The second is a single cubic sumcheck (`cross_layer/projection.rs`) that collapses L per-layer projection sumchecks into one of length $\log L + \log d_{\text{in}}$ by introducing a layer-index variable; this gives a further factor of L saving on the projection transcript. Together with an EVM-targeted Hyrax verifier and a zero-knowledge masking layer, these lay out the path from a working research prototype to an on-chain verifiable inference service.

9 Related Work

Verifiable ML. zkCNN [8] gave the first GKR-based proof system for neural network inference, focused on convolutional networks. [1] extends ZK to proofs of *training* for deep networks. zkLLM [13] adapts a custom IOP to softmax LLM inference using `tlookup`-style structured lookups; zkGPT [10] uses a Plonky2 backend with constraint fusion across operations of a transformer block. Both target the standard softmax + GeLU + LayerNorm stack and pay a significant cost for every non-arithmetic operation. π-Former departs by co-designing the model and proof system: every operation is either a sumcheck or a Lasso lookup by construction, so there is no "ZK-unfriendly operation" tax to pay.

Sumcheck and lookup arguments. The sumcheck protocol underlies modern transparent SNARKs [11, 14]. Lasso provides a practical lookup argument with sub-linear prover cost on structured-decomposable tables [12, 2, 5], and is the natural fit for the additive sub-table structure of π-Former's ϕ. Hyrax [16] gives a transparent, discrete-log-based polynomial commitment with $O(\sqrt{N})$ proofs.

Linear attention and ternary networks. Linear attention [6] replaces softmax with a feature-map kernel and recovers the $O(nd^2)$-vs-$O(n^2d)$ complexity trade-off; [17] projects the keys and values to a low-rank space. BitNet b1.58 [9] introduced ternary weights as a replacement for FP16 dense layers in LLMs, motivated by inference energy on edge devices; π-Former re-purposes the same idea for SNARK proving cost.

10 Conclusion

We presented π-Former, a SNARK system for verifiable transformer inference in which the model architecture and the proof system are co-designed so that every operation in a forward pass maps to a sumcheck claim or a structured table lookup. The architecture combines learnable lookup attention, ternary projections, and a constraint-fused LayerNorm; the proof system batches per-block sub-protocols into six model-level sumchecks, one global Lasso, and one global range proof, with all final Hyrax openings flowing through ten deferred batch accumulators. We implemented the system in $\sim$14 k lines of Rust over BN254, gave formal cross-block batching soundness theorems, and demonstrated end-to-end proving in 4.2 s for a 4-layer, $\sim$924 k-parameter model on a single CPU core. The remaining engineering steps, multi-head, ZK masking, GPU MSM, on-chain verifier, are well-understood and do not change the protocol architecture; we believe π-Former provides a credible foundation for production-grade verifiable transformer inference.

References

[1] Kasra Abbaszadeh, Christodoulos Pappas, Jonathan Katz, and Dimitrios Papadopoulos. Zero-knowledge proofs of training for deep neural networks. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4316–4330, 2024.

[2] Arasu Arun, Srinath Setty, and Justin Thaler. Jolt: Snarks for virtual machines via lookups. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 3–33. Springer, 2024.

[3] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[4] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

[5] Oleg Fomenko and Anton Levochko. Understanding lasso: A novel lookup argument protocol. *Cryptology ePrint Archive*, 2025.

[6] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.

[7] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *Annual International Cryptology Conference*, pages 359–388. Springer, 2022.

[8] Tianyi Liu, Xiang Xie, and Yupeng Zhang. Zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2968–2985, 2021.

[9] Shuming Ma, Hongyu Wang, Shaohan Huang, Xingxing Zhang, Ying Hu, Ting Song, Yan Xia, and Furu Wei. Bitnet b1. 58 2b4t technical report. *arXiv preprint arXiv:2504.12285*, 2025.

[10] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. {zkGPT}: An efficient non-interactive zero-knowledge proof framework for {LLM} inference. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 2045–2063, 2025.

[11] Srinath Setty. Spartan: Efficient and general-purpose zksnarks without trusted setup. In *Annual International Cryptology Conference*, pages 704–737. Springer, 2020.

[12] Srinath Setty, Justin Thaler, and Riad Wahby. Unlocking the lookup singularity with lasso. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 180–209. Springer, 2024.

[13] Haochen Sun, Jason Li, and Hongyang Zhang. zkllm: Zero knowledge proofs for large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4405–4419, 2024.

[14] Justin Thaler. *Proofs, arguments, and zero-knowledge*, volume 4. Foundations and Trends in Privacy and Security, 2022.

[15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[16] Riad S Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zksnarks without trusted setup. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 926–943. IEEE, 2018.

[17] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.